

ComputeModule: Implementing GEMM & ALU in `dependencies_received()`

This guide shows exactly what code goes inside `dependencies_received()` in **compute_module.cpp only** — no `.h` changes needed. It translates the `gemm()` and `alu()` functions from `vta.cc` into standard SystemC/C++.

1. Current State vs. What We Are Adding

Right now, `dependencies_received()` in every module (Load, Compute, Store) looks like this:

```
void ComputeModule::dependencies_received() {
    finish.notify(latency()); // timing only – no actual computation
}
```

This is the timing simulator: it knows *how long* GEMM takes but does not *do* GEMM. The goal is to add the actual loop structure from `vta.cc` before the `finish.notify()` call:

```
void ComputeModule::dependencies_received() {

    if (current->get_name() == "GEMM") {
        // ← GEMM computation goes here (Section 3)
    }
    else if (/* ALU instruction */) {
        // ← ALU computation goes here (Section 4)
    }

    finish.notify(latency()); // always last
}
```

2. Memory Arrays — Defined in compute_module.cpp (NOT in .h)

In HLS, the memory buffers are declared as `static` inside `compute()` or passed as parameters from `vta()`. In SystemC, we put them as **static variables at the top of compute_module.cpp**, before any function. This does not require touching any `.h` file.

```
// — At the top of compute_module.cpp, after the #include lines —————

// These persist between calls to dependencies_received() (just like HLS static)

static uint32_t uop_mem[2048] = {}; // micro-op SRAM (private to Compute)
static int32_t acc_mem[512][16] = {}; // accumulator SRAM [tile][element] (private to Compute)
static int8_t inp_mem[512][16] = {}; // input SRAM [tile][element] (filled by Load)
static int8_t wgt_mem[512][16] = {}; // weight SRAM [tile][element] (filled by Load)
static int8_t out_mem[512][16] = {}; // output SRAM [tile][element] (read by Store)
```

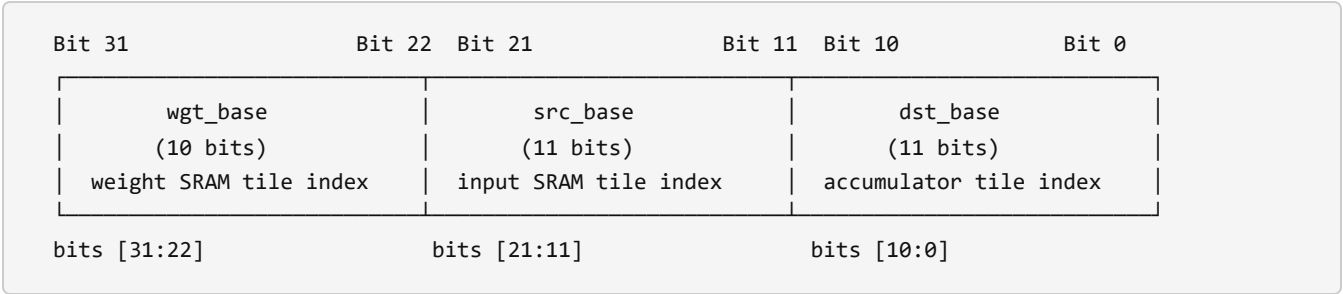
Why static? In HLS, `uop_mem` and `acc_mem` are `static` inside `compute()` — they persist their values between instruction calls (like on-chip SRAM that is never erased between instructions). The `static` keyword at file scope in C++ gives the same effect: the arrays are allocated once and their values survive across multiple calls to `dependencies_received()`.

Simplified layout: The HLS uses a complex packed bus format (`bus_T = ap_uint<512>`) where many elements are packed into one wide word. For SystemC we unpack into simple 2-D arrays: `acc_mem[tile_index][element_index]` . Each tile has 16 elements (`VTA_BLOCK_OUT` or `VTA_BLOCK_IN` = 16 in default VTA config).

3. Understanding the Micro-Op and Bitmasks

Before implementing GEMM/ALU, you must understand the **micro-op (uop)**. It is a 32-bit integer stored in `uop_mem[]`. It encodes three SRAM tile indices packed into one word.

3.1 Bit layout of a GEMM micro-op



3.2 How to extract each field in C++ (replacing HLS `.range()`)

HLS uses Xilinx-specific `ap_uint.range(hi, lo)`. In plain C++ we use `>>` (shift right) and `&` (bitwise AND to mask unwanted bits):

What we extract	HLS (vta.cc)	C++ SystemC equivalent	Why the mask
dst_base bits [10:0] — 11 bits	<code>uop.range(10, 0)</code>	<code>(uop >> 0) & 0x7FF</code>	0x7FF = 0111 1111 1111 (eleven 1s) → keeps only the lowest 11 bits
src_base bits [21:11] — 11 bits	<code>uop.range(21, 11)</code>	<code>(uop >> 11) & 0x7FF</code>	shift brings bit-11 to position 0, then mask keeps only 11 bits
wgt_base bits [31:22] — 10 bits	<code>uop.range(31, 22)</code>	<code>(uop >> 22) & 0x3FF</code>	0x3FF = 0011 1111 1111 (ten 1s) → keeps only the lowest 10 bits

3.3 Visual example with a real number

Suppose `uop_mem[0] = 0x00802001` in hex. In binary:

```
Binary: 00000000100000000010000000000001
Bits:   31                                0

dst_base = bits[10:0] = 0000000001 = 1    (acc tile index)
src_base = bits[21:11] = 00000010000 = 16  (inp tile index)
wgt_base = bits[31:22] = 0000100000 = 32  (wgt tile index)

Extraction:
dst = (0x00802001 >> 0) & 0x7FF = 0x001 = 1
```

```
src = (0x00802001 >> 11) & 0x7FF = 0x010 = 16  
wgt = (0x00802001 >> 22) & 0x3FF = 0x020 = 32
```

The final SRAM index for each operand = base from micro-op + loop offset accumulated so far. The loop offsets are what the outer/inner/factor strides control.

4. GEMM Implementation — Full Code with Line-by-Line Explanation

HLS source (vta.cc gemm()) — what we are translating:

Reads the three SRAM buffers (inp_mem, wgt_mem, acc_mem), runs a triple-nested loop (outer × inner × micro-op), does a dot-product MAC inside each micro-op, and writes results back to acc_mem and out_mem.

4.1 HLS instruction field → SystemC getter

HLS VTAGemInsn field	Getter on Instruction*	What it means
insn.iter_out	current->get_outer_loop_iter()	Number of outer loop iterations
insn.iter_in	current->get_inner_loop_iter()	Number of inner loop iterations
insn.uop_bgn	current->get_range_0()	Start index in uop_mem (micro-op loop)
insn.uop_end	current->get_range_1()	End index in uop_mem (exclusive)
insn.dst_factor_out	current->get_gemm_outer_loop_acc()	How much to advance acc offset per outer iteration
insn.src_factor_out	current->get_gemm_outer_loop_inp()	How much to advance inp offset per outer iteration
insn.wgt_factor_out	current->get_gemm_outer_loop_wgt()	How much to advance wgt offset per outer iteration
insn.dst_factor_in	current->get_gemm_inner_loop_acc()	How much to advance acc offset per inner iteration
insn.src_factor_in	current->get_gemm_inner_loop_inp()	How much to advance inp offset per inner iteration
insn.wgt_factor_in	current->get_gemm_inner_loop_wgt()	How much to advance wgt offset per inner iteration
insn.reset_reg	current->get_reset_out()	If true: zero the accumulator tile after writing (used on bias-load pass)

4.2 The GEMM code (goes inside dependencies_received, inside the "GEMM" branch)

```
// -----
// GEMM: multiply input tiles × weight tiles, accumulate into acc_mem, write out_mem
// Mirrors: gemm() in vta.cc with the three nested loops + tensor MAC
// -----

// STEP 1 – initialise the outer-loop SRAM offsets to zero
// These grow with each outer iteration to select the next group of tiles
int dst_offset_out = 0; // accumulator tile offset (outer loop)
int src_offset_out = 0; // input tile offset (outer loop)
int wgt_offset_out = 0; // weight tile offset (outer loop)
```

```

// STEP 2 – outer loop (mirrors EXE_OUT_LOOP in vta.cc)
for (int it_out = 0; it_out < current->get_outer_loop_iter(); it_out++) {

    // inner offsets start from the current outer offset position
    int dst_offset_in = dst_offset_out;
    int src_offset_in = src_offset_out;
    int wgt_offset_in = wgt_offset_out;

    // STEP 3 – inner loop (mirrors EXE_IN_LOOP in vta.cc)
    for (int it_in = 0; it_in < current->get_inner_loop_iter(); it_in++) {

        // STEP 4 – micro-op loop: iterates over uop_mem[range_0 .. range_1]
        // Each micro-op word encodes one (dst, src, wgt) tile triplet
        for (int upc = current->get_range_0(); upc < current->get_range_1(); upc++) {

            // STEP 5 – read the 32-bit micro-op
            uint32_t uop = uop_mem[upc];

            // STEP 6 – extract the three tile-base indices from the micro-op
            // and add the accumulated loop offsets to get final SRAM indices
            // (replaces HLS uop.range() bitfield extraction)
            int dst_idx = (int)((uop >> 0) & 0x7FF) + dst_offset_in; // acc_mem tile to read/write
            int src_idx = (int)((uop >> 11) & 0x7FF) + src_offset_in; // inp_mem tile to read
            int wgt_idx = (int)((uop >> 22) & 0x3FF) + wgt_offset_in; // wgt_mem tile to read

            // STEP 7 – loop over all 16 output channels in this tile
            for (int oc = 0; oc < 16; oc++) {

                // STEP 8 – load the current accumulator value for this output element
                int32_t accum = acc_mem[dst_idx][oc];

                // STEP 9 – dot product: sum over all 16 input channels
                // mirrors the innermost VTA_BLOCK_IN loop in vta.cc gemm()
                int32_t tmp = 0;
                for (int ic = 0; ic < 16; ic++) {
                    int8_t w = wgt_mem[wgt_idx * 16 + oc][ic]; // weight element [output_ch][input_ch]
                    int8_t x = inp_mem[src_idx][ic]; // input element [input_ch]
                    tmp += (int32_t)(x * w); // multiply-accumulate
                }

                // STEP 10 – add dot product to running accumulator
                accum += tmp;

                // STEP 11 – write back to accumulator SRAM
                // reset_out=true means: zero the tile (used during bias-load pass)
                acc_mem[dst_idx][oc] = current->get_reset_out() ? 0 : accum;

                // STEP 12 – write truncated output to out_mem for Store to pick up
                // HLS: o_tensor[b][oc] = (out_T) accum.range(VTA_OUT_WIDTH-1, 0)
                // In C++: take the low 8 bits of accum
                out_mem[dst_idx][oc] = (int8_t)(accum & 0xFF);
            }
        }
    } // end micro-op loop
}

```

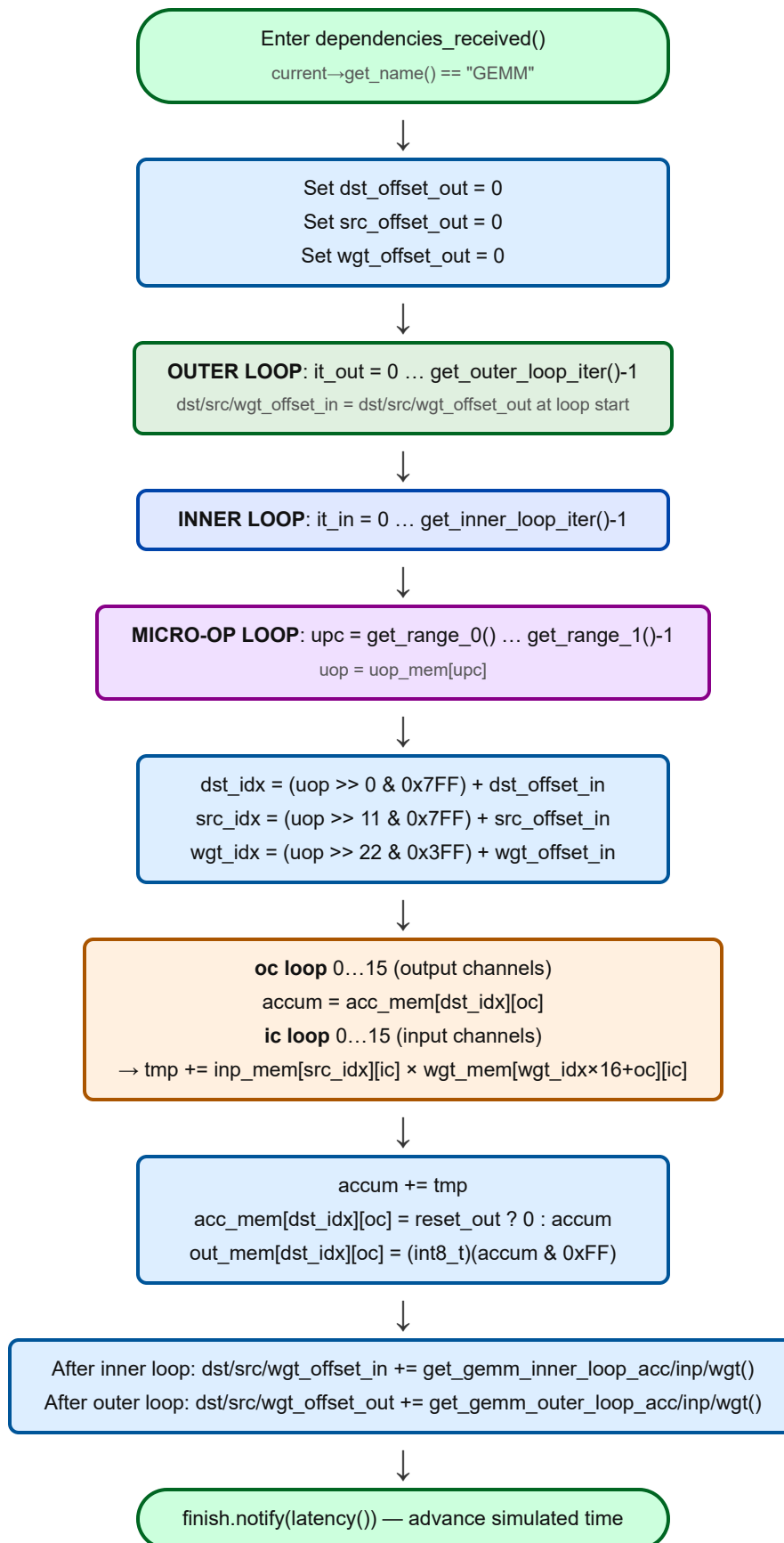
```
// STEP 13 – advance inner offsets by the per-iteration strides
//   HLS: dst_offset_in += insn.dst_factor_in; etc.
dst_offset_in += current->get_gemm_inner_loop_acc();
src_offset_in += current->get_gemm_inner_loop_inp();
wgt_offset_in += current->get_gemm_inner_loop_wgt();

} // end inner loop

// STEP 14 – advance outer offsets by the per-iteration strides
dst_offset_out += current->get_gemm_outer_loop_acc();
src_offset_out += current->get_gemm_outer_loop_inp();
wgt_offset_out += current->get_gemm_outer_loop_wgt();

} // end outer loop
```

5. GEMM Dataflow Flowchart



6. ALU Implementation — Full Code with Line-by-Line Explanation

HLS source (vta.cc alu()) — what we are translating):
Reads two tiles from acc_mem (dst and src), applies an element-wise operation (MAX, MIN, ADD, or SHR), and writes results back to acc_mem and out_mem. No weight memory is involved.

6.1 ALU operations

HLS opcode	Operation	Example use
VTA_ALU_OPCODE_MAX (1)	max(src_0, src_1)	ReLU activation: max(value, 0)
VTA_ALU_OPCODE_MIN (0)	min(src_0, src_1)	Clipping
VTA_ALU_OPCODE_ADD (2)	src_0 + src_1	Residual (skip connection) add
VTA_ALU_OPCODE_SHR (3)	src_0 >> src_1	Fixed-point shift (quantisation scaling)

ReLU example: $\text{ReLU}(x) = \max(x, 0)$. In VTA this is an ALU instruction with opcode MAX and immediate value 0, so src_1 = 0 for every element. The instruction name in the CSV will contain "MAX" or similar.

6.2 HLS instruction field → SystemC getter (ALU)

HLS VTAAIulnsn field	Getter on Instruction*	What it means
insn.iter_out	current->get_outer_loop_iter()	Outer loop count
insn.iter_in	current->get_inner_loop_iter()	Inner loop count
insn.uop_bgn	current->get_range_0()	Micro-op start index
insn.uop_end	current->get_range_1()	Micro-op end index (exclusive)
insn.dst_factor_out	current->get_alu_outer_loop_dst()	acc dst-tile offset stride (per outer iter)
insn.src_factor_out	current->get_alu_outer_loop_src()	acc src-tile offset stride (per outer iter)
insn.dst_factor_in	current->get_alu_inner_loop_dst()	acc dst-tile offset stride (per inner iter)
insn.src_factor_in	current->get_alu_inner_loop_src()	acc src-tile offset stride (per inner iter)

6.3 ALU micro-op bit layout

Bit 31	Bit 22	Bit 21	Bit 11	Bit 10	Bit 0
(unused)	src_base (11 bits) acc source tile index		dst_base (11 bits) acc destination tile idx		

dst_idx = (uop >> 0) & 0x7FF + dst_offset_in ← tile to read AND write

```
src_idx = (uop >> 11) & 0x7FF + src_offset_in  ← tile to read as second operand
(no wgt field for ALU)
```

6.4 The ALU code

```
// -----
// ALU: element-wise operation on accumulator tiles
// Mirrors: alu() in vta.cc – same loop structure, same micro-op decoding,
//          but uses acc_mem for both operands (no inp/wgt)
// -----

// Determine which ALU operation from the instruction name
// (the CSV stores the operation type as part of the instruction name)
std::string name = current->get_name(); // e.g. "ALU", "ALU-MAX", "ALU-ADD", "ALU-SHR"

// STEP 1 – outer-loop offsets
int dst_offset_out = 0; // acc destination tile offset (outer loop)
int src_offset_out = 0; // acc source tile offset      (outer loop)

// STEP 2 – outer loop (mirrors EXE_OUT_LOOP in vta.cc alu())
for (int it_out = 0; it_out < current->get_outer_loop_iter(); it_out++) {

    int dst_offset_in = dst_offset_out;
    int src_offset_in = src_offset_out;

    // STEP 3 – inner loop (mirrors EXE_IN_LOOP in vta.cc alu())
    for (int it_in = 0; it_in < current->get_inner_loop_iter(); it_in++) {

        // STEP 4 – micro-op loop
        for (int upc = current->get_range_0(); upc < current->get_range_1(); upc++) {

            // STEP 5 – read micro-op
            uint32_t uop = uop_mem[upc];

            // STEP 6 – decode dst and src tile indices (ALU has no wgt field)
            int dst_idx = (int)((uop >> 0) & 0x7FF) + dst_offset_in; // tile to write
            int src_idx = (int)((uop >> 11) & 0x7FF) + src_offset_in; // tile as second operand

            // STEP 7 – element-wise operation over all 16 elements in the tile
            for (int oc = 0; oc < 16; oc++) {

                // STEP 8 – read both operands from acc_mem
                int32_t src_0 = acc_mem[dst_idx][oc]; // first operand = destination tile
                int32_t src_1 = acc_mem[src_idx][oc]; // second operand = source tile
                // Note: HLS also supports use_imm (immediate value instead of src_1)
                // We use src_1 from acc_mem here; for ReLU src_idx tile holds zeros

                // STEP 9 – apply the operation
                int32_t result;
                if (name.find("MAX") != std::string::npos) {
                    result = (src_0 > src_1) ? src_0 : src_1; // MAX – used for ReLU
                } else if (name.find("MIN") != std::string::npos) {
                    result = (src_0 < src_1) ? src_0 : src_1; // MIN
                } else if (name.find("ADD") != std::string::npos) {
                    result = src_0 + src_1; // ADD – residual connection
                }
            }
        }
    }
}
```

```
    } else {  
        result = src_0 >> src_1; // SHR - fixed-point scaling  
    }  
  
    // STEP 10 - write result back to acc_mem destination tile  
    acc_mem[dst_idx][oc] = result;  
  
    // STEP 11 - write truncated result to out_mem for Store  
    out_mem[dst_idx][oc] = (int8_t)(result & 0xFF);  
}  
  
} // end micro-op loop  
  
// STEP 12 - advance inner offsets  
dst_offset_in += current->get_alu_inner_loop_dst();  
src_offset_in += current->get_alu_inner_loop_src();  
  
} // end inner loop  
  
// STEP 13 - advance outer offsets  
dst_offset_out += current->get_alu_outer_loop_dst();  
src_offset_out += current->get_alu_outer_loop_src();  
  
} // end outer loop
```

7. Complete dependencies_received() — Ready to Type

This is the full function as it should look in **compute_module.cpp**. Replace the current one-liner with this:

```
void ComputeModule::dependencies_received() {

    std::string name = current->get_name();

    // — GEMM: matrix multiply input x weight → accumulator —————
    if (name == "GEMM") {

        int dst_offset_out = 0, src_offset_out = 0, wgt_offset_out = 0;

        for (int it_out = 0; it_out < current->get_outer_loop_iter(); it_out++) {
            int dst_offset_in = dst_offset_out;
            int src_offset_in = src_offset_out;
            int wgt_offset_in = wgt_offset_out;

            for (int it_in = 0; it_in < current->get_inner_loop_iter(); it_in++) {

                for (int upc = current->get_range_0(); upc < current->get_range_1(); upc++) {
                    uint32_t uop = uop_mem[upc];

                    int dst_idx = (int)((uop >> 0) & 0x7FF) + dst_offset_in;
                    int src_idx = (int)((uop >> 11) & 0x7FF) + src_offset_in;
                    int wgt_idx = (int)((uop >> 22) & 0x3FF) + wgt_offset_in;

                    for (int oc = 0; oc < 16; oc++) {
                        int32_t accum = acc_mem[dst_idx][oc];
                        int32_t tmp = 0;
                        for (int ic = 0; ic < 16; ic++) {
                            int8_t w = wgt_mem[wgt_idx * 16 + oc][ic];
                            int8_t x = inp_mem[src_idx][ic];
                            tmp += (int32_t)(x * w);
                        }
                        accum += tmp;
                        acc_mem[dst_idx][oc] = current->get_reset_out() ? 0 : accum;
                        out_mem[dst_idx][oc] = (int8_t)(accum & 0xFF);
                    }
                }

                dst_offset_in += current->get_gemm_inner_loop_acc();
                src_offset_in += current->get_gemm_inner_loop_inp();
                wgt_offset_in += current->get_gemm_inner_loop_wgt();
            }

            dst_offset_out += current->get_gemm_outer_loop_acc();
            src_offset_out += current->get_gemm_outer_loop_inp();
            wgt_offset_out += current->get_gemm_outer_loop_wgt();
        }

        // — ALU: element-wise operation on accumulator tiles —————
```

```

else if (name != "GEMM" && name != "LOAD ACC" &&
        name != "NOP-COMPUTE-STAGE" && name != "FINISH") {

    int dst_offset_out = 0, src_offset_out = 0;

    for (int it_out = 0; it_out < current->get_outer_loop_iter(); it_out++) {
        int dst_offset_in = dst_offset_out;
        int src_offset_in = src_offset_out;

        for (int it_in = 0; it_in < current->get_inner_loop_iter(); it_in++) {

            for (int upc = current->get_range_0(); upc < current->get_range_1(); upc++) {
                uint32_t uop = uop_mem[upc];

                int dst_idx = (int)((uop >> 0) & 0x7FF) + dst_offset_in;
                int src_idx = (int)((uop >> 11) & 0x7FF) + src_offset_in;

                for (int oc = 0; oc < 16; oc++) {
                    int32_t src_0 = acc_mem[dst_idx][oc];
                    int32_t src_1 = acc_mem[src_idx][oc];
                    int32_t result;

                    if (name.find("MAX") != std::string::npos)
                        result = (src_0 > src_1) ? src_0 : src_1;
                    else if (name.find("MIN") != std::string::npos)
                        result = (src_0 < src_1) ? src_0 : src_1;
                    else if (name.find("ADD") != std::string::npos)
                        result = src_0 + src_1;
                    else
                        result = src_0 >> src_1;

                    acc_mem[dst_idx][oc] = result;
                    out_mem[dst_idx][oc] = (int8_t)(result & 0xFF);
                }
            }

            dst_offset_in += current->get_alu_inner_loop_dst();
            src_offset_in += current->get_alu_inner_loop_src();
        }

        dst_offset_out += current->get_alu_outer_loop_dst();
        src_offset_out += current->get_alu_outer_loop_src();
    }
}

// — NOP / FINISH / LOAD ACC: no computation — fall through —————

finish.notify(latency()); // always last: advance simulation time
}

```

8. What Goes Where — Summary

File	What you add	Where exactly
compute_module.cpp	Five static arrays: uop_mem, acc_mem, inp_mem, wgt_mem, out_mem	After the #include lines, before any function definition
compute_module.cpp	GEMM + ALU loop code	Inside dependencies_received(), before finish.notify()
compute_module.h	<i>Nothing</i>	Professor confirmed: everything needed is already declared

9. Key Things to Remember When Typing the Code

#	Point	Why it matters
1	finish.notify(latency()) is ALWAYS called last	If you put it before the loops, the timing fires before computation — wrong order
2	The bitmasks 0x7FF and 0x3FF are not magic numbers	$0x7FF = 2^{11}-1$ (mask for 11-bit index), $0x3FF = 2^{10}-1$ (mask for 10-bit index). They come from the VTA buffer depth constants.
3	ALU has no wgt_idx — do NOT extract bits [31:22] for ALU	Only GEMM micro-op has a weight field; ALU only has dst and src in the uop
4	The else branch catches all ALU types (MAX, MIN, ADD, SHR)	The latency() function also uses the same "else = ALU" pattern — they must match
5	acc_mem keeps its values between instructions	Because it's static — the accumulator from instruction N is still there when instruction N+1 runs. This is how GEMM accumulates across multiple passes.
6	out_mem written by Compute, read by Store	In a full implementation, Store would need access to out_mem — for now it's in compute_module.cpp file scope